# [1편] 인공지능 첫걸음

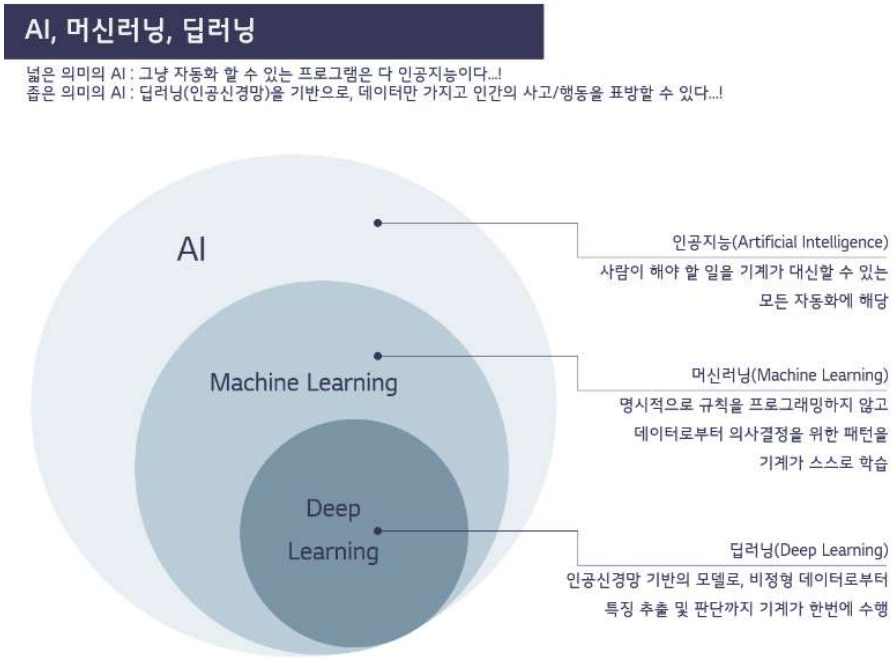
## 1. 인공지능(Artificial Intelligence;AI)이란

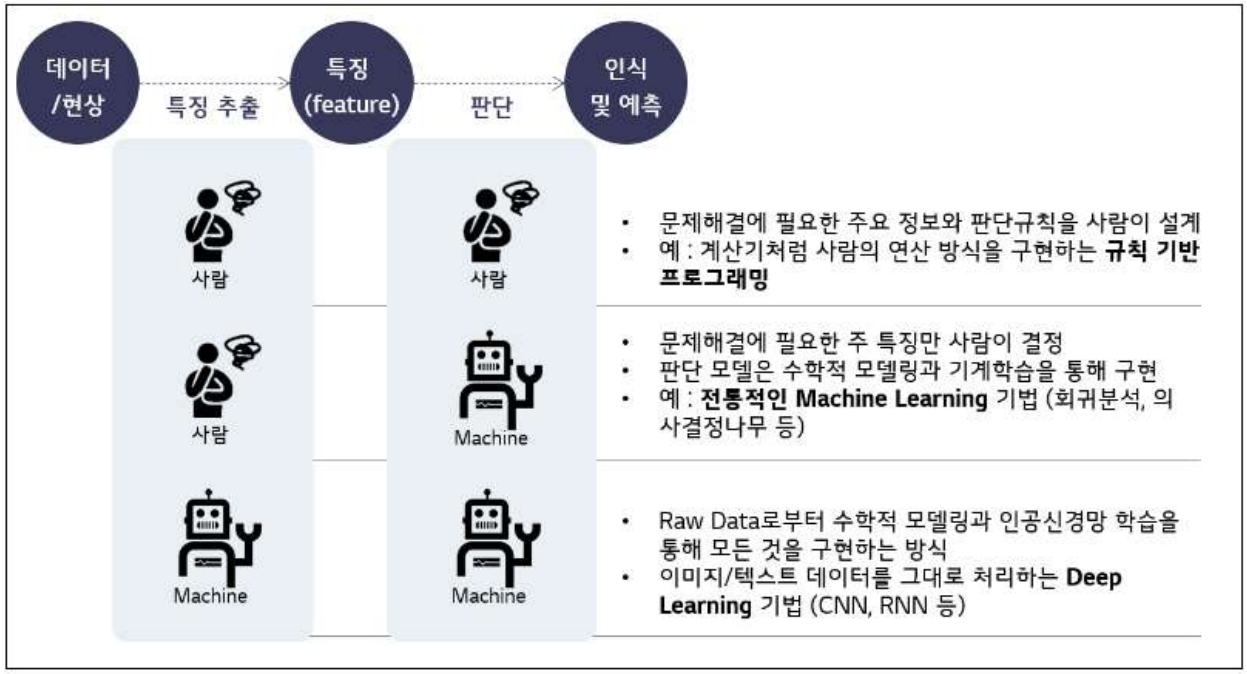
1) 강 인공지능 (Strong AI, General AI)
인간과 사랑에 빠질 수있을 정도로 감정적으로 풍부한 인공지능이라든지, 만능 가사 비서 로봇, 아니면 인간에게 아주 위협적인 형태가 될 수도 있는 영화상의 인공지능은 '강 인공지능'입니다.

2) 강 인공지능 (Strong AI, General AI)
반면 우리의 일상생활에서 만나는 모든 인공지능은 '약 인공지능'입니다.
약인공지능은 제한된 환경에서 구체적인 특정 업무를 수행하는 데 있어서 사람과 비슷한, 또는 사람 이상의 성능을 낼 수 있는 인공지능입니다.

## 2. 넓은 의미의 AI, 좁은 의미의 AI

[ 특징(feature) 추출과 의사 결정 ]
기계자동화(규칙 기반의 AI)와 머신러닝, 딥러닝 방식을 구분하는 기준에는 두 가지가 있습니다.
첫 번째로 특징 추출입니다. 특징 추출은 문제 해결을 위해 어떤 정보가 유용할 지, 중요한 것을 꺼내는 과정입니다.
두 번째로 판단 방식입니다. 추출한 중요 특징으로부터 인식을 하거나 판단을 모델을 만드는 단계입니다.



# [2편] 기계도 사람처럼 판단할 수 있을까?

## 1. 머신러닝과 딥러닝

'딥러닝'은 '머신러닝'이라고 불리는 기계학습의 한 분야로, 기계가 비정형 데이터를 입력받은 후 데이터의 주요한 특징을 알아서 추출하고 이를 바탕으로 의사결정을 하는 기술입니다.

- 머신러닝이 다루는 정형 데이터(structured data)
  딥러닝이 아닌, 전통적인 머신러닝 기반의 기술들로 다룰 수 있는 데이터는 주로 정형데이터입니다.
  정형데이터는 안정성은 높으나 체계적으로 구조가 고정되어있어 유연하지는 않은 데이터타입입니다.
  머신러닝 기반의 기술이란 빅데이터 분석 기법들이라고 생각하시면 됩니다.

- 딥러닝이 다루는 비정형 데이터(unstructured data)
  반면 딥러닝 기반의 AI가 다루는 데이터는 비정형 데이터입니다.
  비정형 데이터는 사람이 따로 예쁘게 양식을 정리해놓지 않은, 다양한 형식을 가지는 데이터입니다.

# [3편] 시각을 얻은 인공지능

1. Convolutional Neural Network(CNN)

이미지 처리에 특화된 신경망, CNN(Convolutional Neural Network)
CNN은 이미지로부터 특징을 추출하는 (1) Feature Extraction 영역과, 특정 태스크 수행을 위해 덧붙이는 (2) 태스크 수행 영역 두 가지로 구성

- 특징 추출(Feature Extraction)
  이미지로부터 특징을 추출하는 역할은 컨볼루션(Convolution) 연산과 풀링(Pooling) 연산이 수행합니다.
  컨볼루션 연산은 컨볼루션 필터(또는 커널)가 입력 이미지를 상하좌우로 훑으며 주요한 특징이 있는지 찾아내는 과정입니다.
  컨볼루션 필터를 여러 개 설정하면 그만큼 이미지로부터 다양한 특징을 찾아낼 수 있습니다.
  이렇게 찾아낸 결과 특징을 Feature map(또는 convolved feature)라고 부릅니다.
  컨볼루션 필터가 이미지를 상하좌우로 훑으며 특징을 찾아낸 뒤, 이 결과로부터 정보를 추리는 풀링 연산이 이어집니다.
  풀링 연산은 Feature map을 역시 상하좌우로 훑으며 핵심적인 정보만을 영역별로 샘플링 하는데요,
  주로 영역 내 가장 큰 값만을 남기고 나머지 값을 버리는 MaxPooling 방식을 적용합니다.

- 태스크 수행
  이미지로부터 주요 특징을 찾아냈다면 이 정보를 활용하여 목표로 하는 태스크를 수행해야 합니다.
  이 부분의 구조나 필요한 연산의 종류는 하고자 하는 태스크의 종류에 따라 다릅니다.

  [ Classification ]
  입력으로 받은 이미지를 지정된 K개의 클래스(또는 카테고리) 중 하나로 분류하는 과제입니다.
  강아지와 고양이를 분류하거나, 공장에서 제품 사진을 보고 양호/불량을 판별하는 업무 등에 쓰일 수 있습니다.

  [ Detection ]
  입력으로 받은 이미지에서 특정 개체가 어디에 위치하는지, (x, y) 좌표값을 찾아주는 과제입니다.
  스마트폰으로 사진을 찍을 때 자동으로 인물의 얼굴에 네모박스를 치고 포커스하는 기능을 본 적 있으시죠?
  이 경우 얼굴을 detect하는 기능이 탑재되어 있겠네요.

  [ Segmentation ]
  Detection이 네모 박스로 개체를 찾아준다면 Segmentation은 조금 더 정밀하게, 픽셀 단위로 영역을 구별해줍니다.

# [4편] 언어를 이해하는 인공지능

## 1. 언어를 인식하는 인공지능

자연어이해(NLU; Natural Language Understanding)

[ 자연(Natural) ]
자연어(Natural Language; 自然語)란, 사람들이 일상적으로 쓰는 언어로, 기원을 찾기 힘들며 자연 발생한다는 특징이 있습니다.
예를 들어 우리가 쓰는 한국어나 영어, 중국어, 일본어 등이 그 예입니다.
반대되는 개념으로는 인공어(Constructed Language; 人工語)가 있습니다.
이는 의도와 목적에 따라 인공적으로 만든 언어로, 프로그래밍 언어와 같은 기계어라든가 전지구인의 공용 언어 사용을 위해 만들어진 에스페란토 등이 이에 해당합니다.

[ 언어(Language) ]
'언어'라는 데이터 타입이 갖는 특성은 무엇일까요?
언어는 말하고 듣는 음성과, 쓰고 읽는 문자로 이루어져 있습니다.

[ 이해(Understanding) ]
이해한다는것은 처리한다는 것보다 한단계 더 높은 수준을 요구합니다.
처리는 기계적으로 규칙을 따라 얼마든지 수행 가능하겠지만 이해는 맥락 내용에 대한 파악을 전제로 합니다.

요즘은 자연어 처리(NLP)나 자연어 이해(NLU)라는 용어를 거의 구분없이 사용하고는 있습니다.
엄밀히 나누자면 NLP 안에 NLU가 포함됩니다.
NLP에만 해당하는 부분은 언어의 형식, 구문론과 관련된 기능들입니다.

## 2. 기계에게 사람의 언어를 인식시키려면?

[ Tokenizing(Parsing) ]
한 덩이로 되어있는 문장을 인공신경망에 인식시키기 위해서 세부 단위로 쪼개는 작업
- 어절, 형태소, 음절, 자소

[ 워드임베딩(word embedding) ]
쪼개진 토큰들은 인공신경망이 계산할 수 있도록 벡터로 바꾸어줘야 하는데요, 토큰을 벡터화하는 것을 워드임베딩이라고 부릅니다.
- 원-핫 인코딩(One-hot Encoding)
  토큰을 벡터화하는 가장 쉽고 직관적인 방법은 우리가 알고있는 모든 토큰들을 쭈루룩 줄세워 사전을 만들고, 순서대로 번호를 붙이는 작업
  원-핫 인코딩 방식은 아주 간단하긴 합니다만 큰 문제가 있습니다.
  하지만 원-핫 인코딩 방식은 아주 간단하긴 합니다만 큰 문제가 있습니다.
  토큰이 다양하고 수가 많을수록 토큰 하나를 표현하기 위해서 굉장히 길이가 긴 벡터를 필요
- CBOW와 SKIPGRAM
  CBOW 방식은 인공지능에게 문장을 알려주되 중간중간 빈칸을 만들어 들어갈 단어(토큰)을 유추시킵니다.
  SKIPGRAM 방식은 인공지능에게 단어(토큰) 하나를 알려주고, 주변에 등장할만한 그럴싸한 문맥을 만들도록 시킵니다.

[ 다양한 자연어이해 과제들 ]
- 문장/문서 분류(Sentence/Document Classification) : 입력받은 텍스트를 지정된 K개의 클래스(또는 카테고리) 중 하나로 분류하는 과제(예, 사용자 긍/부정 리뷰)
- Sequence-to-Sequence : 문장/또는 문서를 입력으로 받아 문장을 출력하는 과제(예, 번역)
- 질의 응답(Question Answering) : 예, 상담 챗봇 및 콜센터
  사용자 질문이 들어오면 내가 가진 매뉴얼 내에서 가장 답변이 될 가능성이 높은 영역을 리턴하는 MRC(Machine Reading Comprehension)
  가장 유사한 과거 질문/답변(FAQ)를 꺼내 주는 IR(Information Retrieval) 형태가 있습니다.

# [5편] 과거의 경험을 통해 현재를 배우는 인공지능

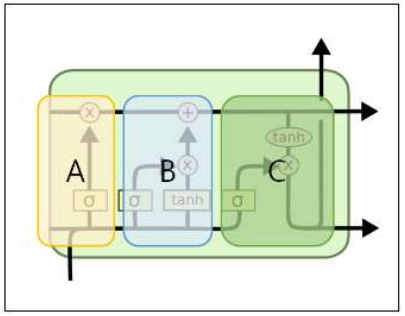
## 1. 시간 흐름에 따른 데이터(Sequential data) 처리하기

1) Recurrent Neural Network(RNN; 순환 신경망)
과거의 처리 이력(경험 지식 등)을 압축하여 반영하는 RNN
과거에 데이터를 처리하여 결과를 출력했던 과정의 일부를 가져와 현 시점에서 데이터를 처리하고 결과를 출력하는 데 도움을 준다.

[ 장점 ]
- RNN은 시간 흐름에 따른 과거 정보를 누적할 수 있다
- RNN은 가변 길이의 데이터를 처리할 수 있다
- RNN은 다양한 구성의 모델을 만들 수 있다

[ 단점 ]
- 연산 속도가 느리다
- 학습이 불안정하고 어렵다
  Gradient Exploding : 학습해야 할 값이 폭발적으로 증가하는 현상이 발생
  Gradient Vanishing : timestep이 길어지면 저 멀리 있는 과거의 이력은 현재의 추론에 거의 영향을 미치지 못하는 문제
- 실질적으로 과거 정보를 잘 활용할 수 있는 모델이 아니다
  RNN의 장기 종속성/의존성 문제(Long-term dependency)

2) LSTM(Long-short term memory)
RNN의 단점을 보안하고자 정보 흐름을 잘 조절하기 위해 성능을 개선한 특별한 형태의 뉴런




Gate라 하는 부분은 정보의 흐름을 조절하는 관문 역할을 수행합니다.

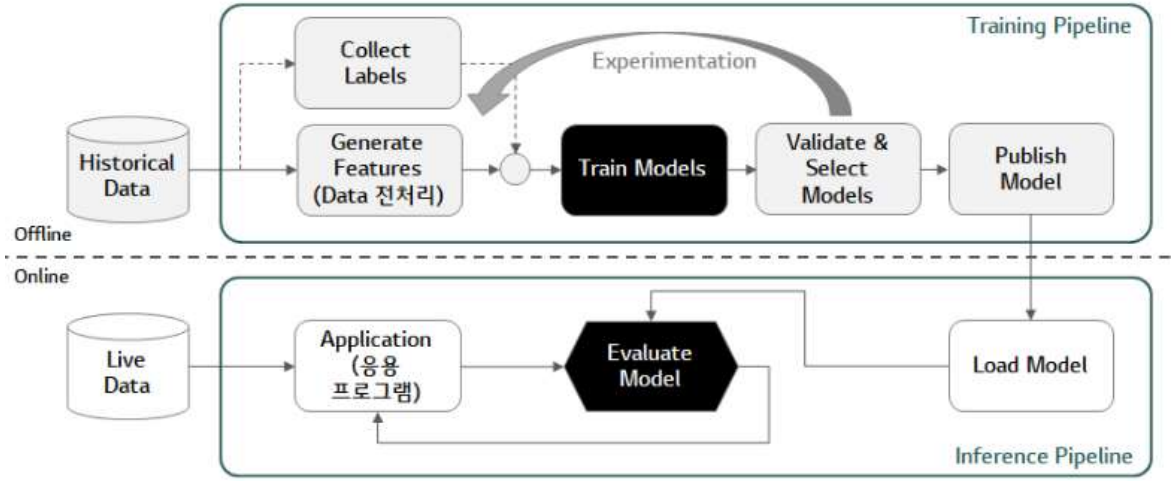
A 부분은 forget gate라 불리며, 말 그대로 잊어버림에 대한 조절
B 부분은 input gate로, 현재의 정보(input data)를 얼마나 반영할지를 결정
C 부분은 output gate로, 현재 시점에 연산된 최종 정보를 다음 시점에 얼마나 넘길지를 결정

LSTM에는 이 세 가지의 gate가 있어서, 정보의 흐름을 인공지능이 자체적으로 더 원활하게 조절하는 기능을 합니다.
따라서 데이터가 길어진다고 해도 일반 RNN에 비해 더 좋은 예측을 할 수 있게 됩니다.
다만 계산 과정이 복잡한 만큼 연산속도는 조금 더 느려지는 단점이 있습니다.
그래서 이를 조금 개선한 것이 GRU(Gated Recurrent Unit)

# [6편] 헛똑똑이 인공지능 제대로 가르치기

1. AI Process

AI를 도입하는 과정은 크게 오프라인 프로세스와 온라인 프로세스로 나뉩니다.
각 과정은 고객사 프로젝트에서 오픈을 위해 열심히 준비하는 개발 과정과, 고객사 오픈 이후의 운영 환경에 해당하는 과정이라고 볼 수 있습니다.




1) Offline Process
오프라인 프로세스는 과거에 만들어진 데이터(historical data)를 가공하는 것에서부터 시작(그 때 그 때 발생하는 실시간 데이터가 아닌, DB 등에 이미 수집되어 있는 데이터)
AI 모델러는 기 확보된 데이터를 확인하고 정제하여 필요한 부분을 취하거나(Generate features) 필요한 경우 라벨을 붙입니다(Collect labels).
이 때의 라벨이란 AI 모델 학습을 위해 필요한 정보로, 인공지능이 맞춰야 하는 '정답'이라고 생각하시면 됩니다.
데이터를 마련했으면 어떤 머신러닝/딥러닝 알고리즘을 활용하여 모델을 학습할 것인지 정하고,
좋은 성능을 달성할 때까지(Validate & Select models) 반복실험을 진행합니다(Train models).
보통은 실험의 결과를 진단하고 이 때 발견한 문제점을 해결하거나 더 개선된 성능을 낼 수 있도록 실험(Experimentation)을 반복하는데, 이 과정을 튜닝이라고 합니다.
튜닝은 모델 학습에 필요한 여러 수치 설정값(하이퍼파라미터)을 조절하는 등의 역할을 포함
여러 실험을 반복하며 개선된 성능의 모델은 배포할 수 있도록 최종 선택(Publish model)됩니다.
여기까지의 과정은 더 나은 AI모델을 만들기 위해 모델을 학습시키는 과정으로, Training Pipeline이라고 합니다.

2) Online Process
AI 모델은 오프라인 프로세스에서 배울 것을 모두 배웠으니, 실전 환경에 뛰어들어 추론(Inference)을 해야 합니다.
모델 추론을 포함한 온라인 프로세스의 대부분은 머신러닝/딥러닝이라기보다는 개발 영역(Application)에 가깝습니다.
이제부터 모델이 처리할 데이터는 기존에 정리하여 모아놓은 데이터가 아닙니다.
실시간으로 들어오는, 운영 환경의 스트리밍 데이터(Live Data)입니다. AI 모델은 과거의 학습 지식을 바탕으로 현장의 데이터를 추론하게 됩니다.

3) 오버피팅(Overfitting)과 일반화 성능(Generalization)
Generalization이라고 부르는, 일반화 성능과 관련된 개념입니다. Generalization의 정의는 '이전에 본적 없는 데이터'에 대해서도 잘 수행하는 능력입니다.
훈련시에만 잘 작동하고 일반화 성능이 떨어지는 모델을 오버피팅(Overfitting)되었다고 합니다.

3) 오버피팅(Overfitting)과 일반화 성능(Generalization)
Generalization이라고 부르는, 일반화 성능과 관련된 개념입니다. Generalization의 정의는 '이전에 본적 없는 데이터'에 대해서도 잘 수행하는 능력입니다.
훈련시에만 잘 작동하고 일반화 성능이 떨어지는 모델을 오버피팅(Overfitting)되었다고 합니다.

4) Training, Validation, Test
확보한 데이터(Historical data)를 Training, Validation, Test의 세 set으로 나누고 각 set이 수행할 역할을 구분
엄격히 비율이 정해져 있는 것은 아니지만, 보통은 Training, Validation, Test set을 8:1:1, 6:2:2 정도로 구분

[ Training set ]
Training set은 머신러닝/딥러닝 모델을 학습하는 데 이용하는 데이터입니다.
모델은 Training set의 입력 데이터와 정답을 보고, 정답을 더 잘 맞추기 위해 노력합니다.
이는 단순히 최적화(Optimization)에 해당합니다.

[ Validation set ]
Validation set은 머신러닝/딥러닝 모델에게 정답을 알려 줄 데이터는 아니지만, 우리가 모델을 튜닝하는 데 도움을 주는 데이터
즉 모델의 일반화 성능을 판단하여 이어질 실험을 계획하는 데 이용
모델은 Validation set의 정답은 본 적이 없지만 이 set의 입력 데이터만으로 일단 정답을 추론
모델이 배우지 않았던 데이터, 즉 처음 보는 데이터에 대해 얼마나 잘 맞추는 지를 계산할 수 있으니 이 결과를 보고 우리는 실험을 개선
이를 통해 다음 실험시 오버피팅을 방지할 전략을 세울 수 있습니다.

[ Test set ]
Test set은 모델의 학습에 어떤 식으로도 전혀 관여하지 않는 데이터로, 오로지 모델의 최종 성능을 평가하기 위해 따로 떼어놓은 데이터
여러 모델간 성능을 비교할 땐 Test set에 대한 스코어를 활용
Training set으로 모델을 학습하고, Validation set에 대한 성능을 확인하며 모델을 개선해왔으니 그 결과를 공정히 평가하기 위한 기준

5) 학습 곡선(Learning curve) 확인하기
역할에 따라 데이터를 나누었다면 학습이 제대로 이루어지는지, 일반화 성능이 떨어지거나 하여 오버피팅이 발생하지는 않았는지 확인
학습 곡선이란 학습이 진행됨에 따라 모델의 성능을 기록하는 그래프(개선하고 있고, Validation set에 대해서는 별다른 향상이 없는 시점이 오버피팅 발생 시점)
오버피팅이 발생했는지 확인할 수 있는 방법은 학습 곡선상에서 Training set와 Validation set에 대한 모델의 성능이 어떻게 변화하는지를 확인₩Training set에 대해서만 성능을

6) Regularization
오버피팅을 피하기 위한 모든 전략들을 Regularization이라고 합니다.(Regularization의 목적은 Generalization, 즉 일반화 성능을 향상)
Training set에 대해 더 잘 맞추고자 하는 것이 아닌, 현장에 나가 처음 보는 데이터에 대해서도 잘 맞출 수 있도록 하는 목적

- 데이터 증강(Data Augmentation)
  오버피팅을 피하는 가장 좋은 방법은 데이터를 더 많이 확보

- Capacity 줄이기
  Capacity는 모델의 복잡한 정도를 나타내는 개념(오버피팅의 경향이 발견된다 싶으면 내 모델의 층 수를 줄여본다든지, 한 층의 뉴런 수를 줄인다든지 등의 조치)

- 조기 종료(Early stopping)
  말 그대로 오버피팅이 감지될 경우 목표하는 학습 시간이 다 되지 않았다고 하더라도 '조기 종료'(불필요하게 학습되는 경우를 방지)

- 드롭아웃(Dropout)
  드롭아웃은 학습 과정(Training pipeline)에서 일정 비율 p 만큼의 노드(인공 뉴런)를 무작위로 끄고 진행하는 Regularization 기법
  딥러닝 모델 학습 시 굉장히 많이 적용하는 Regularization 기법 중 하나로 일부 노드가 사라진 상태에서 남아있는 노드만으로 정답을 맞춰야 하는 인공지능 모델은 훨씬 더 강력

# [7편] 다시쓰고 바꿔쓰자! 인공지능 재활용하기

## 1. 쉽지 않은 인공지능 적용하기

1) 구체적이지 않으며 불명확한 태스크
두루뭉실하고 막연한 요구사항보다는 구체적이고 명확한 목표를 세우고 필요한 하위 기능을 쪼개어 생각

2) 적은 데이터, 낮은 품질의 데이터

3) 다른 도메인 환경
오버피팅을 피하고 General하게 만든 모델이라고 해도 추론 환경이 달라지면 성능이 감소하기 마련

## 2. Transfer Learning : 한번 만든 인공지능 모델 우려먹기

'전이 학습'으로도 불리는 Transfer Learning은 한 번 만들어진 딥러닝 모델을 재활용하여 쓸 수 있는 기법
비슷한 태스크를 다른 도메인에 적용할 때, 그리고 그 태스크를 위한 학습 데이터가 부족한 경우 유용하게 쓰일 수 있습니다.
Transfer Learning은  하나의 모델이 이미 배워놓은 지식은 잘 유지하면서도, 새로운 태스크에 대해 필요한 지식을 추가로 습득할 수 있도록 합니다.

## 3. Catastrophic forgetting : 치명적인 기억상실!

딥러닝 모델이 새로운 정보를 학습할 때 이전에 배웠던 정보를 완전히 잊어 버리는 경향이 발생할 수 있는데, 이를 Catastrophic forgetting이라고 합니다.
Catastrophic forgetting이 발생하지 않도록 하면서도 새로운 데이터를 잘 배우기 위해서는 다음과 같은 기법을 활용

[ 더 나은 Transfer Learning을 위한 방법 ]
일반적인 딥러닝 모델은 두 층(layer) 이상의 인공신경망으로 구성
이 신경망은 초반에는 데이터의 구체적이고 기본적인 특징을, 후반부로 갈수록 특정 태스크를 위한 추상적이고 개념적인 특징을 학습
후반부의 신경망 층에 대해서만 파라미터를 학습하고, 전반부의 파라미터는 학습되지 않도록 고정해놓는 기법을 적용할 수 있습니다.
이를 레이어 동결(Layer freezing)이라 부릅니다.

이외에 층마다 Learning rate(학습률)의 차별을 두는 것도 한 방법입니다. (Discriminative fine-tuning)
Learning rate이란 한 번에 인공신경망의 파라미터를 얼마만큼 업데이트 시킬지에 대한 정도로, 사람이 설정하는 값(hyperparameter)

[ Transfer Learning 모델 이용 ]
Transfer Learning은 보통 오픈도메인 데이터에 대해 만들어놓은 모델을 특정 도메인의 태스크에 적용하는 식으로 활용합니다.
일반적인 내용을 두루두루 넓게 학습해놓은 모델의 사전 지식을 구체적인 하위 영역에 활용

[ 컴퓨터 비전에서의 Transfer Learning ]
이미지를 입력 데이터로 처리하는 경우라면 이미지넷 데이터로 학습된 모델에 Transfer Learning을 적용하여 좋은 성능을 낼 수 있습니다.
이미지넷 데이터 인식 대회에서 우승을 했던 GoogleNet(2014 우승)이나 ResNet(2015 우승) 등은 지금까지도 널리 활용되는 기본 모델

[ 자연어 이해(NLU)에서의 Transfer Learning ]
텍스트란 특히나 함축적인 데이터 타입이기 때문에 Transfer Learning을 활용하는 것이 매우 효과적
위키백과와 같은 방대하고 일반적인 지식에 대해서 학습한 적이 있는 모델이라면 일반적인 어휘의 의미를 대강 알고 있는 모델이라고 볼 수 있습니다.
이런 모델이 공개되어있다면 Transfer Learning에 활용하는 것이 좋겠지요.
자사에서 만들어 공개한 KorQuAD(Korean Question Answering Dataset)는 한국어 위키백과를 소스로 한 오픈도메인 질의응답 데이터

# [8편] 준비된 인공지능, Pre-trained AI

## 1. Pre-training: 니가 뭘 요청할지 몰라서 일단 다 공부해놨어

하지만 전이 학습을 염두에 두고 다방면으로 활용할 수 있는 모델을 미리 만들어놓을 수 있습니다.
여러 태스크에 활용하기 위해 여러 지식을 미리 두루두루 학습해놓은 인공지능을 만드는 것이지요.
이러한 학습을 Pre-training, 또는 사전학습이라고 하며 이러한 용도의 모델을 Pre-trained Model, 즉 사전학습 모델이라고 합니다.
사전학습은 보통 특정 데이터타입에 대한 일반적인 지식을 두루 배워놓는 것이 목표

## 2. 대규모 데이터에 대한 Pre-training

어떤 후속 태스크에 적용하든 잘 수행할 수 있도록 하려면 방대한 양의 지식을 골고루 배워놓는 것이 좋습니다.

[ 시각 데이터에 대한 사전학습 ]
이미지넷 데이터 인식 대회는 가장 유명한 이미지 사전학습 과제

[ 언어 데이터에 대한 사전학습 ]
언어에 대해 전반적으로 배워놓는다는 것은 문맥에 따라 활용되는 단어의 의미, 뉘앙스, 적절한 문체 등을 습득
자연어의 경우 이미지나 비디오 데이터와는 달리, 언어권의 차이로 인해 데이터를 언어권별로 각각 수집하여 학습시켜야 하는 문제가 있습니다.
하지만 무난하게 활용하기 좋은 데이터로는 위키피디아(Wikipedia)가 있습니다.
이외에 뉴스, 리뷰 등의 데이터도 자주 활용되는 사전학습용 데이터

## 3. Self-Supervised Learning: 나 혼자 어떻게든 해볼게

하지만 대규모로 구할 수 있는 데이터라 해도 라벨까지 잘 달려있는 경우는 드뭅니다.
라벨(label)이라 함은 데이터에 대해 인공지능이 예측하기를 희망하는 결과

Self-Supervised Learning은 사람이 만들어주는 정답 라벨이 없어도 기계가 시스템적으로 자체 라벨을 만들어서 사용하는 학습 방법

예: Google BERT(Bidirectional Encoder Representations from Transformers)
Self-Supervised Learning 기법으로 사전학습을 하고 다양한 태스크에 Transfer Learning을 할 수 있는 대표적인 예로 Google의 'BERT'라는 모델이 있습니다.
BERT는 자연어처리 연구 패러다임을 전환한 계기가 된 모델입니다. 인공지능에서의 자연어 처리는 BERT 이전과 이후로 나뉘죠.
예전부터 사전학습과 Transfer Learning의 개념이 있긴 했으나, 기존의 사전학습이 워드 임베딩 등 그저 보조적 역할을 수행하는 느낌이었다면 BERT는 사전학습 자체가 주가 되는 모델입니다.

# [9편] 족집게 데이터로 인공지능 학습하기

## 1. 데이터의 바다, 정보의 홍수

기술도 데이터는 많으나 인공지능을 '학습시킬 데이터'를 마련하기 쉽지 않을 때 이용할 수 있는 기술
Active Learning은 라벨링을 할 수 있는 인적 자원은 있지만, 많은 수의 라벨링을 수행할 수 없을 때 효과적으로 라벨링을 하기 위한 기법

### 1) Active Learning: 족집게 데이터로 공부하기

**Motive**

모델이 잘 맞추기 어려운 데이터를 찾아 학습한다면,
더 적은 훈련시간으로 더 좋은 성능을 낼 수 있을 것이다.

**Objective**

Labeling을 위한 예산이 한정되었을 때,
모델의 성능을 극대화할 수 있는 Labeling 대상 데이터를 찾기

Active Learning은 학습 데이터 중 모델 성능 향상에 효과적인 데이터들을 선별한 후, 선별한 데이터를 활용해 학습을 진행하는 방법
효과적인 데이터를 선별하는 방법을 연구하는 것이 Active Learning

### 2) Active Learning의 절차

Active Learning은 크게 4단계로 구성
- Training a Model : 초기 학습 데이터(labeled data)를 이용해 모델을 학습
- Select Query : 라벨이 되지 않은 데이터 풀로부터 모델에게 도움이 되는 데이터를 선별
- Human Labeling : 선별한 데이터를 사람이 확인하여 라벨을 태깅
- 선별한 라벨 데이터를 기존 학습 데이터와 병합한 후, 다시 모델을 학습

### 3) Query Strategy: 이 데이터를 제게 가르쳐 주십시오!

Active Learning의 핵심은 성능 향상에 효과적인 데이터를 선별하는 방법이며 이러한 데이터 선별 방법을 '쿼리 전략(Query Strategy)'
쿼리 전략을 어떻게 정하느냐에 따라서 선별할 데이터가 달라집니다.
- 학습된 모델의 판정 값을 기반으로 뽑는 Uncertainty Sampling
- 여러 개의 모델을 동시에 학습시키면서 많은 모델이 틀리는 데이터를 선별하는 Query by committee
- 데이터가 학습 데이터로 추가될 때, 학습된 모델이 가장 많이 변화하는 데이터를 선별하는 Expected Impact
- 데이터가 밀집된 지역의 데이터들을 선별하는 Density weighted method
- 데이터들을 최대한 고르게 뽑아서 전체 분포를 대표할 수 있도록 데이터를 선별하는 Core-set approach

[ Uncertainty Sampling ]
Uncertainty Sampling은 가장 단순한 쿼리 전략입니다. AI 모델은 가장 불확실하다(least certain)고 생각하는 데이터를 추출하여 라벨링이 필요하다고 요청하게 됩니다.

[ Query by committee ]
Query by committee는 여러 AI 모델간의 의견불일치를 종합 고려하는 방식. 여러 모델간 추론한 결과 불일치가 많은 데이터일수록 가장 헷갈리는 데이터, 즉 라벨링을 진행할 대상

# [10편] 뭣이 중한지 알아보는 인공지능

## 1. 긴 입력 데이터 처리하기

RNN은 시간 순서에 따른 데이터를 처리하는 인공신경망(누적된 데이터가 입력될 경우 먼 과거의 내용을 잘 반영하기 어렵다)

## 2. 어텐션 메커니즘(Attention mechanism)

어텐션 메커니즘이란 인공신경망이 입력 데이터의 전체 또는 일부를 되짚어 살펴보면서 어떤 부분이 의사결정에 중요한지, 중요한 부분에 >>집중<<하는 방식

1) 어텐션 스코어(Attention score)
중요한 단어에 집중한다는 것은 어텐션 스코어를 계산
어텐션 스코어는 인공신경망 모델이 각 인코딩 timestep마다 계산된 특징(feature)를 가지고 자동으로 계산하는 0~1사이의 값

2) 컨텍스트 벡터(Context vector)
어디를 더 살펴보고 어디는 대충 볼지에 대해 어텐션 스코어를 구하고 나면 현재 디코딩할 단어와의 관련성을 반영하여 다시 입력 문장을 인코딩
이는 중요도에 따라 전체 문맥의 정보를 잘 반영하고 있다고 하여 컨텍스트 벡터(Context vector)라고 부릅니다.
중요한 것은 스코어 계산에 필요한 수식이 아니라, 어텐션 메커니즘이 매번 디코딩마다 직전 단계의 벡터 뿐 아니라 과거의 모든 데이터의 특징(feature)들을 고려한다는 점
또 하나의 포인트는, 딥러닝 모델이 스스로 집중할 영역을 파악

## 3. XAI로서의 어텐션

어텐션 메커니즘은 기계가 판단시 중요하게 생각하는 부분을 우리에게 알려주는 역할도 합니다.
설명가능한 인공지능(eXplainable AI;XAI)으로서의 기능을 수행
이를 해석가능한 인공지능(interpretable AI)라고도 부릅니다.
해당 결정을 내릴 때 어떤 부분에 집중해서 판단했는지를 시각화해 보여줄 수 있습니다.

- 텍스트에서의 어텐션
- 이미지에서의 어텐션

## 4. Attention 전성시대, Transformer

이 어텐션이라는것이 어찌나 좋은지, 요즘에는 어텐션만으로 이루어진 인공신경망 구조가 새로 등장

트랜스포머(Transformer)라는 인공신경망은 입력 데이터끼리의 self-attention을 통해 상호 정보교환을 수행하는 것이 특징
문장 내의 단어들이 서로서로 정보를 파악하며, 나와 내 주변 단어간의 관계, 문맥을 더 잘 파악

순차적 계산이 필요 없기 때문에 RNN보다 빠르면서도 맥락 파악을 잘하고, CNN처럼 일부씩만을 보는 것이 아니고 전 영역을 아우릅니다.
하지만 이해력이 좋은 대신에 모델의 크기가 엄청 커지며 고사양의 하드웨어 스펙을 요구한다는 단점이 있는데요, 이러한 한계를 보완하기 위한 다양한 경량화 방안이 연구되고 있습니다.

# [11편] 스스로 진화하는 인공지능, AutoML

1. 사람의 손을 필요로 하는 인공지능
튜닝이라 함은 현재 실험의 결과 양상을 보고 문제점을 진단하고, AI 모델을 조금 더 나은 방향으로 만들고자 실험을 개선
실험을 진행하는 사람의 탄탄한 이론 배경과 더불어 경험과 노하우까지 풍부해야만 불필요한 실험의 반복 횟수를 줄일 수 있지요.

2. 스스로 진화하는 인공지능, AutoML
AutoML(Automated Machine Learning), 말 그대로 자동화된 기계학습
[ AutoML의 역할 ]
- AI 모델을 학습하기 위해 데이터로부터 중요한 특징(feature)을 선택하고 인코딩하는 방식에 대한 Feature Engineering 자동화
- AI 모델 학습에 필요한 사람의 설정들, 하이퍼파라미터를 자동으로 탐색
- AI 모델의 구조 자체를 더 효율적인 방향으로 찾아주는 아키텍쳐 탐색

1) 하이퍼파라미터 탐색 자동화
딥러닝 모델 학습에 필요한 하이퍼파라미터는 다양한 종류있으며 기존에 이미 여러 가지 하이퍼파라미터의 조합을 찾고자 하는 시도가 있었습니다.
자주 쓰이는 것 두 가지만 들자면 그리드 서치(Grid search)와 랜덤 서치(Random search) 방식이 있습니다.
- 그리드 서치 방식은 최적화할 하이퍼파라미터의 값 구간을 일정 단위로 나눈 후, 각 단위 조합을 테스트하여 가장 높은 성능을 낸 하이퍼파라미터 조합을 선택하는 방식
- 랜덤 서치 방식은 랜덤하게 하이퍼파라미터의 조합을 테스트하는 방식인데, 그리드서치에 비해 비교적 빠르게 최적의 조합을 찾아내곤 합니다.

AI모델이 좋은 성능을 달성하기 위한 최적의 하이퍼파라미터의 조합을 찾아주는 Meta Learner가 별도로 있습니다.
Meta Learner는 대부분 RNN과 강화학습을 활용하여 최적의 하이퍼파라미터를 탐색

2) 아키텍처 탐색 자동화
아키텍처는 모델을 이루는 구조를 말하는데, 사람이 어떤 방식으로 모델 구조를 짤지 생각하지 않아도 자동 탐색을 통해 최적 구조를 찾을 수 있습니다.
특히 딥러닝 모델의 경우에는 인공신경망을 활용하기 때문에, NAS(Neural Architecture search)라고 부릅니다.
NAS의 경우도 마찬가지로 대부분 Meta Learner와 Learner로 이루어져 있어서, Learner가 본 과제를 수행하는 AI 모델이라면
Meta Learner가 어떤 구조의 신경망을 만들면 좋은지, 아키텍쳐 구성을 고민

3. AutoML 특징
AutoML을 활용시 사람이 구조를 고민하고, 하이퍼파라미터를 튜닝할 필요 없이 최적의 환경을 결정

4. AutoML 서비스
AWS, Azure, GCP(Google Cloud Platform)와 같은 CSP 업체는 모두 일종의 AutoML 서비스를 제공
단 하이퍼파라미터 선택이나 모델 구조 선택, 이미지/텍스트 관련 태스크별 지원 기능에 차이가
있으므로 원하는 기능이 제공되는지 살펴보고 이용

# [12편] 설명 가능한 인공지능, XAI

1. 종종 이해할 수 없는 결정을 내리는 AI

2. 설명 가능한 인공지능, XAI(eXplainable Artificial Intelligence)
1) XAI의 필요성
인공지능에게 설명력을 부여하는 방법에 대한 연구 분야를 XAI(eXplainable AI), 설명 가능한 인공지능이라고 합니다.
XAI는 모델에 설명 가능한 근거와 해석력을 부여해서 투명성, 신뢰성을 확보하고자 하는 것이 목적

3. XAI를 위한 접근법
첫 번째로, 기존 AI 모델에 설명할 수 있는 어떤 모듈을 덧붙이는 방식

1) 어텐션 메커니즘(Attention Mechanism)을 활용한 XAI
어텐션 메커니즘은 XAI로서의 기능을 수행합니다.
어텐션 메커니즘은 딥러닝 모델이 어떤 결정을 내릴 때, 입력 데이터의 어떤 부분에 집중해서 판단했는지를 시각화해 보여줄 수 있습니다.

2) 설명하는 법 학습하기(Learn to explain)
이 방식은 판단을 하는 딥러닝 모델에 RNN 모듈 등을 덧붙여 인간이 이해할 수 있는 방식의 설명을 생성하도록 하는 방식
하지만 설명 생성에 한계가 있어 널리 활용되는 방식은 아닙니다.

두 번째로, 애초에 설명력있는 모델을 만드는 방법

세 번째로는 인공신경망처럼 복잡한 블랙박스 모델의 일부분을 설명해 줄 수 있는 다른 모델을 활용하여 유추하는 방식